Exercise Sheet 02

The Loot Split

House rules.

- One file per exercise, in `my_work/`: `ex1_purse_rule.py`, `ex2_purse_run.py`, …
- Start each file by typing the **Given** lines. Then write the rest yourself.
- Run it (`python my_work/ex1_purse_rule.py`) and compare with the **expected output**, character by character.
- Indentation is not decoration in Python: the lines *inside* an `if` or a `for` are the ones indented by 4 spaces.
- Stuck for more than 10 minutes? Cheat sheet first, then skip and come back later.
- Hand in: commit and push to `lesson-02`, then open a pull request. Tell Lars which raven was the most annoying.

Exercise 1 — One purse decision

10 min · `ex1_purse_rule.py`

> **RAVEN #201, from Captain Siv, quest master:** *"The rules for the Queen's purse, as Her Majesty wrote them on a napkin at the feast: nothing for novices (fewer than 3 quests done). Otherwise a quarter of the reward on top for difficulty 4 or 5, a tenth for a 3, nothing below that. Can Python read a napkin?"*

GIVEN

```
name = "Wren Hawkridge"
quests_done = 14
difficulty = 4
reward = 3100
```

(a) Use `if` / `elif` / `else` to set a `purse_rate` *and* a short `reason` text. Check for novices first.

(b) Print the purse in crowns and the reason in brackets.

(c) Now change the Given values one at a time: quests_done 2, difficulty 3, difficulty 1. Does the reason change the way you expect? Put the values back afterwards.

EXPECTED OUTPUT

```
Wren Hawkridge: royal purse 775.00 cr (difficulty 4 or 5)
```

Hint: Only the **first** condition that is true runs. Order matters: if the difficulty check came first, a novice on a Wyrm hunt would get paid.

Exercise 2 — The purse run

15 min · `ex2_purse_run.py`

> **RAVEN #202, from Captain Siv:** *"Lovely. Now do it for every quest leader this week, please. In Excel I'd drag the formula down. What's the Python for 'drag down'?"*

GIVEN

```
records = "Wren Hawkridge/3100/4;Bram Ironfoot/3650/3;Thea Yarrow/2800/5;Jory
Pike/2450/2;Ysolde Tallow/4100/4"

for record in records.split(";"):
    name, reward_text, difficulty_text = record.split("/")
```

The two Given lines at the bottom give you one quest leader per trip through the loop: `record.split("/")` cuts `"Wren Hawkridge/3100/4"` into three pieces and puts them in three variables. (Nobody is a novice this time.)

(a) Inside the loop: convert the texts to numbers, work out the rate (a quarter for 4+, a tenth for 3, else 0) and print one line per hero: name left-aligned in 14 characters, purse right-aligned in 10 with two decimals and a thousands separator.

(b) Keep a running `total`. Where must `total = 0` go: inside or before the loop? Try both and watch what happens.

(c) After the loop, print a line of 24 dashes and the total in the same layout.

EXPECTED OUTPUT

```
Wren Hawkridge    775.00
Bram Ironfoot     365.00
Thea Yarrow       700.00
Jory Pike           0.00
Ysolde Tallow   1,025.00
------------------------
Total           2,865.00
```

Exercise 3 Twelve days of supplies

10 min · `ex3_supplies.py`

> **RAVEN #203, from Pell, the Quartermistress:** *"A party of four is marching twelve days into the Northern Wilds. I need the supplies day by day, with a running total. The Grimoire answered 'Run-time error 6: Overflow', which I think means 'a lot'."*

GIVEN

```
heroes = 4
rations_per_hero = 27.50
mule_and_lamp_oil = 25.75
```

(a) Work out the cost per day (rations for every hero, plus the mule and lamp oil).

(b) Loop over the days with `range(1, 13)`. Each time, add the day's cost to a running total and print the line. Day numbers are right-aligned in 2 characters.

(c) After the loop, print the total for the march.

EXPECTED OUTPUT

```
Day  1: supplies 135.75 cr  total so far 135.75 cr
Day  2: supplies 135.75 cr  total so far 271.50 cr
Day  3: supplies 135.75 cr  total so far 407.25 cr
Day  4: supplies 135.75 cr  total so far 543.00 cr
Day  5: supplies 135.75 cr  total so far 678.75 cr
Day  6: supplies 135.75 cr  total so far 814.50 cr
Day  7: supplies 135.75 cr  total so far 950.25 cr
Day  8: supplies 135.75 cr  total so far 1,086.00 cr
Day  9: supplies 135.75 cr  total so far 1,221.75 cr
Day 10: supplies 135.75 cr  total so far 1,357.50 cr
Day 11: supplies 135.75 cr  total so far 1,493.25 cr
Day 12: supplies 135.75 cr  total so far 1,629.00 cr
Supplies for the march: 1,629.00 cr
```

Hint: `range(1, 13)` gives 1 up to **and not including** 13. Same rule as slicing.

Exercise 4 Reward bands

10 min · `ex4_reward_bands.py`

> **RAVEN #204, from Queen Aldara, at dusk:** *"How many quests on the board are in each band? Copper is under 250 crowns, silver under 600, gold under 1,500, dragon is the rest. And what's the top reward? Don't tell me which quest. Actually do. No, don't. A."*

GIVEN

```
rewards = "240,480,700,2600,3150,1340,120,260"
```

(a) Loop over the rewards (`rewards.split(",")` gives you one text at a time; convert it). Print each reward and its band.

(b) Count how many land in each band with four counters, and track the highest reward seen so far.

(c) After the loop, print the counts and the highest reward.

EXPECTED OUTPUT

```
240 cr -> copper
480 cr -> silver
700 cr -> gold
2,600 cr -> dragon
3,150 cr -> dragon
1,340 cr -> gold
120 cr -> copper
260 cr -> silver
Copper: 2, silver: 2, gold: 2, dragon: 2
Highest reward: 3,150 cr
```

Hint: Because `elif` only runs when the earlier checks were false, "under 600" doesn't need to say "and 250 or more". Python already knows.

Exercise 5 How many months?

10 min · `ex5_wyrm_bounty.py`

> **RAVEN #205, from Orrin of the Clockworks:** *"Hypothetically. The bounty on the Wyrm of the Northern Wilds is 2,800 crowns, and the Queen raises it 3 % every month nobody slays it. How many months until it's over 3,500? Asking for a friend. The friend is Biscuit."*

GIVEN

```
bounty = 2800
target = 3500
raise_percent = 3
```

(a) You don't know how many times to loop, so use `while bounty < target`. Each time: raise the bounty, count the month, print it.

(b) After the loop, print how many months it took.

(c) Set `raise_percent = 0` and run it. Press `Ctrl+C` when you've seen enough. Then put the 3 back. Congratulations, you've met the infinite loop.

EXPECTED OUTPUT

```
Month 1: 2,884.00 cr
Month 2: 2,970.52 cr
Month 3: 3,059.64 cr
Month 4: 3,151.42 cr
Month 5: 3,245.97 cr
Month 6: 3,343.35 cr
Month 7: 3,443.65 cr
```

```
Month 8: 3,546.96 cr
Over 3,500 cr after 8 months.
```

Exercise 6 Free rations

10 min · ex6_rations.py

> **RAVEN #206, from Pell:** *"Every member eats free in the guild kitchens for 25 days a year, plus 1 after 2 years in the guild, plus 3 after 5, plus 5 after 10 (not added up: the best one applies). The Scriptorium writes 'gone' for those who've retired to a farm. Skip those; Dalvric's version gave them 30 days and a farewell feast."*

GIVEN

```
this_year = 2025
joined = "2024,2021,gone,2019,2015,gone,2009"
```

(a) Loop over the entries. If an entry is `"gone"`, use `continue` to jump to the next one.

(b) Otherwise work out the years in the guild and the days (`days += 5` is short for `days = days + 5`), and print the line.

(c) Make it say "1 year" rather than "1 years". Small things matter on a notice.

EXPECTED OUTPUT

```
Joined 2024: 1 year in the guild -> 25 days of free rations
Joined 2021: 4 years in the guild -> 26 days of free rations
Joined 2019: 6 years in the guild -> 28 days of free rations
Joined 2015: 10 years in the guild -> 30 days of free rations
Joined 2009: 16 years in the guild -> 30 days of free rations
```

Exercise 7 Dalvric's guild calendar STRETCH

15 min · ex7_guild_calendar.py

> **RAVEN #207, from Pell:** *"Dalvric's calendar spell: a patrol every 3rd day, Biscuit's bath every 5th, and a feast on the 15th and 30th (which are bath days too, so it says both). Days with nothing on are left out. It was the only thing of his that worked. Then it stopped."*

GIVEN

```
for day in range(1, 31):
```

(a) Inside the loop decide the label. Think about the **order** of the checks: day 15 must say "FEAST DAY + bath", not "patrol". Use `continue` for days with nothing on.

(b) Then, with a `while` loop that counts up from 21, find the first feast day after day 20 and print it.

EXPECTED OUTPUT

```
Day  3: patrol
Day  5: Biscuit's bath
Day  6: patrol
Day  9: patrol
Day 10: Biscuit's bath
Day 12: patrol
Day 15: FEAST DAY + bath
Day 18: patrol
Day 20: Biscuit's bath
Day 21: patrol
Day 24: patrol
Day 25: Biscuit's bath
Day 27: patrol
Day 30: FEAST DAY + bath
First feast day after day 20: day 30
```

Hint: `day % 15 == 0` is true on the 15th and 30th. Programmers call this exercise FizzBuzz. Now you know the secret handshake.

Bug hunt Pages from the Grimoire, page 2

15 min · `bughunt/purse_bug.py`

Dalvric's purse run. When Captain Siv ran it, *everyone* got a purse, including novices on their first rat hunt, and the total came to minus forty crowns. There are **three** bugs: one crashes, one pays the wrong people, one wrecks the total even when every line is right. The rules are in the file header.

EXPECTED OUTPUT ONCE FIXED

```
Wren Hawkridge    775.00
Jory Pike           0.00
Bram Ironfoot     365.00
Biscuit             0.00
------------------------
Total           1,140.00
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.